

1 A Reference

1.1 misc

```
1 #pragma GCC optimize("Ofast,no-stack-protector,unroll-loops,
   fast-math")
2 #pragma GCC target("sse,sse2,sse3,ssse3,sse4,popcnt,abm,mmx,avx,
   ,tune=native")
3 cin.tie(0)->sync_with_stdio(false);
4 stringstream ss; string s;
5 getline(cin, s);
6 ss << s;
7 #include <cctype> <cmath> <cstdio> <cstdlib> <cstring> <map> <
   queue> <stack> <set> <string> <utility> <vector>
8 __int128
9 /*
10 g++ -Wshadow -Wall -O2 file.cpp -g
11 g++ -x c++ -Wall -std=c++17 -O2 -static -pipe -o "$DEST" "$@"
12 gdb a.out
13 b main : breakpoint at main
14 n : next one line (skip function)
15 s : next one line (into function)
16 c : continue
17 * bt : list current stack
18 up : go to the up stack
19 down : go to the down stack
20 */
21 default_random_engine gen(time(NULL));
22 uniform_int_distribution<ll> unif(0, (1l)1e18);
23 uniform_real_distribution<double> unif(0, 1);
24 #include <ext/pb_ds/assoc_container.hpp>
25 using namespace __gnu_pbds;
26 typedef tree<int, null_type, less<int>, rb_tree_tag,
   tree_order_statistics_node_update> indexed_set;
27 // Less(set) or Less_equal(multiset)
28 indexed_set s;
29 s.insert(2), s.insert(3), s.insert(7), s.insert(9);
30 cout << *s.find_by_order(2) << " // 7
31 cout << s.order_of_key(7) << "\n"; // 2
32 cout << s.order_of_key(6) << "\n"; // 2
33 cout << s.order_of_key(8) << "\n"; // 3
34 cout << __builtin_popcount(10) << "\n";
```

1.2 python

```
1 arr = list()
2 arr = [0] * n # fixed-size list
3 arr.sort()
4 str.isdigit() # islower, isupper, isalpha, isdecimal, isspace,
   isalnum
5 str.lower() # upper
6 str = str[::-1] # reverse
7 str = str[:i] + str[i + 1:] # string erase s[i]
8 dic = {} # for adjacency list, dic[i] = list()
9 set = set()
10 set.add(1)
11 set.remove(1)
12 1 in set # True or False
13 for i in range(start, end, dif): # for(int i = start; i < end;
   i += dif)
```

2 B Graph Theory

2.1 BCC Tarjan

```
1 int dfn[N], low[N]; bool cut[N];
2 int link_comp[M]; bool used[M];
3 // If |comp|=1, it is bridge
4 vector<pair<int, int>> adj[N]; // {neighbor, egdeIndex}
5 stack<int> sta;
6 int ord = 1, comp = 0;
7 void DFS(int x, int p = -1) {
8     dfn[x] = low[x] = ord++;
9     int child_cnt = 0;
10    for(auto [v, i] : adj[x]) {
11        if(v == p) continue;
12        if(!dfn[v]) {
13            sta.push(i);
14            child_cnt++; used[i] = true;
15            DFS(v, x);
16            low[x] = min(low[v], low[x]);
17            if((dfn[x] != 1 && dfn[x] <= low[v]) || (dfn[x] ==
18                1 && child_cnt >= 2)) {
19                cut[x] = true;
20                int top_idx = -1;
21                comp++;
22            }
23        }
24    }
```

```
21 while(top_idx != i) {
22     link_comp[sta.top()] = comp;
23     top_idx = sta.top();
24     sta.pop();
25 }
26 }
27 } else {
28     if(!used[i]) {
29         sta.push(i);
30         used[i] = true;
31     }
32     low[x] = min(dfn[v], low[x]);
33 }
34 }
35 if(dfn[x] == 1) {
36     if(!sta.empty()) {
37         comp++;
38         while(!sta.empty()) {
39             link_comp[sta.top()] = comp;
40             sta.pop();
41         }
42     }
43 }
44 }
```

2.2 Bipartite Matching

```
1 int match[N]; bool used[N]; vector<int> adj[N];
2 bool DFS(int x) {
3     for(auto u : adj[x]) {
4         if(used[u]) continue;
5         used[u] = 1;
6         int next = match[u];
7         if(next == -1 || DFS(next)) {
8             match[u] = x;
9             return 1;
10        }
11    }
12    return 0;
13 }
14 int n, m; /* Left size and right size */
15 int Bipartite_match() {
16     memset(match, -1, sizeof(match));
17     memset(used, 0, sizeof(used));
18     int match_number = 0;
19     for(int i = 1; i <= n; i++) {
20         if(DFS(i)) {
21             memset(used, 0, sizeof(used));
22             match_number++;
23         }
24     }
25     // and if(match[i] != -1) {i(right set) and match[i](left
   set) match}
26     return match_number;
27 }
28 void add_edge(int left_node, int right_node)
29 { adj[left_node].push_back(right_node); }
```

2.3 Euler Path

```
1 typedef pair<int, int> Edge;
2 vector<Edge> adj[N];
3 vector<int> Euler_path;
4 // Store Node or Edge
5 int deg[N], n;
6 bool vis_node[N], vis_edge[M];
7 void DFS(int x) {
8     vis_node[x] = 1;
9     while(!adj[x].empty()) {
10        auto [v, e] = adj[x].back();
11        adj[x].pop_back();
12        if(!vis_edge[e]) {
13            vis_edge[e] = 1;
14            DFS(v);
15            Euler_path.push_back(v);
16        }
17    }
18 }
19 void Euler() {
20     DFS(1);
21     Euler_path.push_back(1);
22     for(int i = 1; i <= n; i++) {
23         if(!vis_node[i] && deg[i] != 0) || deg[i] % 2 != 0)
24             // No Euler Path
25     }
26 }
```

2.4 Kuhn Munkres

```

1 struct KM { // 0-base, maximum matching
2     ll w[N][N], hl[N], hr[N], slk[N];
3     int fl[N], fr[N], pre[N], qu[N], ql, qr, n;
4     bool vl[N], vr[N];
5     void init(int _n) {
6         n = _n;
7         for (int i = 0; i < n; ++i) fill_n(w[i], n, -INF);
8     }
9     void add_edge(int a, int b, ll wei)
10    { w[a][b] = wei; }
11    bool Check(int x) {
12        if (vl[x] = 1, ~fl[x]) return vr[qu[qr++]] = fl[x] = 1;
13        while (~x) swap(x, fr[fl[x] = pre[x]]);
14        return 0;
15    }
16    void bfs(int s) {
17        fill_n(slk, n, INF), fill_n(vl, n, 0), fill_n(vr, n, 0);
18        ql = qr = 0, qu[qr++] = s, vr[s] = 1;
19        for (ll d;;) {
20            while (ql < qr)
21                for (int x = 0, y = qu[ql++]; x < n; ++x)
22                    if (!vl[x] && slk[x] >= (d = hl[x] + hr[y] - w[x][y])) {
23                        if (pre[x] = y, d) slk[x] = d;
24                        else if (!Check(x)) return;
25                    }
26            d = INF;
27            for (int x = 0; x < n; ++x)
28                if (!vl[x] && d > slk[x]) d = slk[x];
29            for (int x = 0; x < n; ++x) {
30                if (vl[x]) hl[x] += d;
31                else slk[x] -= d;
32                if (vr[x]) hr[x] -= d;
33            }
34            for (int x = 0; x < n; ++x)
35                if (!vl[x] && !slk[x] && !Check(x)) return;
36        }
37    }
38    ll solve() {
39        fill_n(fl, n, -1), fill_n(fr, n, -1), fill_n(hr, n, 0);
40        for (int i = 0; i < n; ++i)
41            hl[i] = *max_element(w[i], w[i] + n);
42        for (int i = 0; i < n; ++i)
43            bfs(i);
44        ll res = 0;
45        for (int i = 0; i < n; ++i)
46            res += w[i][fl[i]];
47        // i match fl[i]
48        return res;
49    }
50 };

```

2.5 König's theorem

node cover = select some node cover every edge.

In bipartite graph,

minimum node cover = maximum matching.

maximum independent set = n - minimum node cover.

A set of minimum node cover can be found by this way.

1. Decide the direction of Edge.

Matched Edge $L \rightarrow R$

Unmatch Edge $L \leftarrow R$

2. DFS the vertex in L which is unvisited

3. The set of minimum node cover is

The unvisited vertex in L

The visited vertex in R

2.6 Max Clique

```

1 struct Maximum_Clique {
2     typedef bitset<N> bst;
3     bst N[N], empty;
4     int p[N], n, ans;
5     void BronKerbosch2(bst R, bst P, bst X) {
6         if (P == empty && X == empty)
7             return ans = max(ans, (int)R.count()), void();
8         bst tmp = P | X;
9         int u;
10        if ((R | P | X).count() <= ans) return;
11        for (int uu = 0; uu < n; ++uu) {
12            u = p[uu];
13            if (tmp[u] == 1)
14                break;
15        }
16        // if (double(clock())/CLOCKS_PER_SEC > .999)
17        // return;
18        bst now2 = P & ~N[u];
19        for (int vv = 0; vv < n; ++vv) {

```

```

20            int v = p[vv];
21            if (now2[v] == 1) {
22                R[v] = 1;
23                BronKerbosch2(R, P & N[v], X & N[v]);
24                R[v] = 0, P[v] = 0, X[v] = 1;
25            }
26        }
27    }
28    void init(int _n) {
29        n = _n;
30        for (int i = 0; i < n; ++i)
31            N[i].reset();
32    }
33    void add_edge(int u, int v) { N[u][v] = N[v][u] = 1; }
34    int solve() { // remember srand
35        bst R, P, X;
36        ans = 0, P.flip();
37        for (int i = 0; i < n; ++i)
38            p[i] = i;
39        random_shuffle(p, p + n), BronKerbosch2(R, P, X);
40        return ans;
41    }
42 };

```

2.7 Topological sort

```

1 void DFS(int x) {
2     vis[x] = 1;
3     for (auto i : adj[x]) {
4         if (vis[i] == 1) cyclic = 1;
5         if (!vis[i]) DFS(i);
6     }
7     ord.push_back(x); vis[x] = 2;
8 }
9 void Topological_sort() {
10    for (int i = 1; i <= n; i++)
11        if (!vis[i]) DFS(i);
12    if (cyclic) printf("IMPOSSIBLE");
13    else // reverse ord is order
14 }

```

3 C Dynamic Programing

3.1 Classic DP Problem Set

```

1 "Bounded Knapsack"
2 for (int i = 0; i < n; i++) { // for each item and Quantity[i]
3     copy
4     for (int k = 1; k <= quantity[i]; k <= 1) {
5         quantity[i] -= k;
6         item.push_back(Item(k * weight[i], k * value[i]));
7     }
8     if (quantity[i] > 0) item.push_back(Item(quantity[i] *
9         weight[i], quantity[i] * value[i]));
10 }
11 // and use 01-Knapsack to solve problem
12 "01 and Unbounded Knapsack"
13 for (int i = 0; i < n; i++) {
14     for (int j = x; j - weight[i] >= 0; j--) { // if this is
15         infinite Knapsack, j = (weight[i] ~ x)
16         dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
17     }
18 }
19 ans = dp[x]
20 "Sum of Subset"
21 sum[0] = true;
22 for (int i = 0; i < n; i++) {
23     for (int j = MAX; j >= coin[i]; j--) {
24         sum[j] |= sum[j - coin[i]];
25     }
26 }
27 "LCS"
28 for (int i = 1; i <= (int)a.size(); i++) {
29     for (int j = 1; j <= (int)b.size(); j++) {
30         if (a[i - 1] == b[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
31         else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
32     }
33 }
34 ans = dp[a.size()][b.size()]
35 "LIS"
36 for (int i = 0; i < n; i++) {
37     auto it = lower_bound(dp.begin(), dp.end(), arr[i]);
38     if (it == dp.end()) dp.push_back(arr[i]);
39     else *it = arr[i];
40 }
41 ans = dp.size()
42 "Removal Game"

```

```

40 ll solve(ll l, ll r) { //calculate left ~ right interval, the
    first player can get how many score
41     if(l > r) return 0;
42     if(l == r) return arr[l];
43     if(l == r - 1) return max(arr[l], arr[r]);
44     if(dp[l][r]) return dp[l][r];
45     ll ans1, ans2;
46     ans1 = min(arr[l] + solve(l + 2, r), arr[l] + solve(l + 1, r
        - 1));
47     ans2 = min(arr[r] + solve(l + 1, r - 1), arr[r] + solve(l, r
        - 2));
48     return dp[l][r] = max(ans1, ans2);
49 }
50 "Cutting Stick"
51 int solve(int l, int r) {
52     if(r - l <= 1) return 0;
53     if(dp[l][r]) return dp[l][r];
54     ll ans = INF, len = cut[r] - cut[l];
55     for(int i = l + 1; i < r; i++) {
56         ans = min(ans, len + solve(l, i) + solve(i, r));
57     }
58     return dp[l][r] = ans;
59 }
60 void Cutting_stick() {
61     cut[0] = 0, cut[n + 1] = 1; // and input cut[1 ~ n]
62     ans = solve(0, n + 1);
63 }

```

3.2 Digit DP

```

1 vector<int> v;
2 ll dp[L][D][2][2];
3 ll DFS(int level, int digit, bool lead_zero, bool limit) {
4     if(level == -1) {
5         return 1;
6     }
7     if(dp[level][digit][lead_zero][limit]) {
8         return dp[level][digit][lead_zero][limit];
9     }
10    int up = limit ? v[level] : 9;
11    ll res = 0;
12    for(int i = 0; i <= up; i++) {
13        // digit != i is the problem condition
14        // digit is the last digit
15        if(digit != i || (lead_zero && i == 0)) {
16            res += DFS(level - 1, i, lead_zero && i == 0, limit
                && (i == v[level]));
17        }
18    }
19    return dp[level][digit][lead_zero][limit] = res;
20 }
21 ll solve(ll d) {
22     v.clear();
23     v = itov(d);
24     /* ex. d = 1234
25      * v = {4, 3, 2, 1} */
26     memset(dp, 0, sizeof(dp));
27     return DFS((int)v.size() - 1, 11, 1, 1);
28 }

```

3.3 Subset DP

```

1 /* Give weight of each person and the maximum allowed weight in
    the elevator.
2  * Find the minimum number of elevator rides
3  * dp[subset] = {minimum rides, the last rides weight}*/
4 dp[0] = {1, 0};
5 for(int s = 1; s < (1 << n); s++) {
6     dp[s] = {INF, INF};
7     for(int i = 0; i < n; i++) {
8         if(s & (1 << i)) {
9             pair<ll, ll> option = dp[s ^ (1 << i)];
10            if(option.second + weight[i] > x) {
11                option.first++;
12                option.second = weight[i];
13            } else {
14                option.second += weight[i];
15            }
16            dp[s] = min(dp[s], option);
17        }
18    }
19 }

```

3.4 Sum Over Subsets

```

1 // dp[mask] = sigma of dp[i](i in mask)
2 for(int k = 0; k < K; k++) { // K = 20

```

```

3     for(int s = 0; s < (1 << K); s++) {
4         if(s & (1 << k)) dp[s] += dp[s ^ (1 << k)];
5     }
6 }

```

4 Combination

4.1 Burnside's Lemma

Burnside's Lemma count the number of distinct combinations so that symmetric combinations are count only once.

$$\frac{1}{n} * \sum_{k=1}^n c(k)$$

where there n ways to change the position of a combinations, and there are c(k) combinations that remain unchanged when the kth way is applied(ex. rotate k steps clockwise).

Example :

Calculate the number of necklace of pearls, where each pearls has m possible colors. Two necklaces are symmetric if they are similar after rotating them.

The solution is

$$\frac{1}{n} * \sum_{k=0}^{n-1} m^{gcd(k,n)}$$

4.2 Catalan Number

Define C_n is n-th Catalan number

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-i-1}$$

or

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

Catalan number is represent for

1. The number of valid parenthesis expression of length is $2n$
2. In $n \times n$ grid, the number of path from $(0, 0)$ to (n, n) and always below $(x = y)$ this line
3. The number of distinct binary tree of n nodes
4. The number of way that split convex of $(n + 2)$ vertex into triangle

4.3 Fast Matrix Exponentiation

```

1 // a_i=a_i-1+a_i-2+a_i-3+a_i-4+a_i-5+a_i-6
2 using Matrix = array<array<ll, N>, N>;
3 Matrix matrix_muti(Matrix x, Matrix y, ll p) {
4     Matrix c = zero();
5     for(int i = 0; i < n; i++) {
6         for(int j = 0; j < n; j++) {
7             for(int k = 0; k < n; k++)
8                 c[i][j] = (c[i][j] + x[i][k] * y[k][j]) % p;
9         }
10    }
11    return c;
12 }
13 Matrix fp(Matrix x, ll y, ll p) {
14     Matrix res = I();
15     while(y) {
16         if(y & 1) res = matrix_muti(x, res, MOD);
17         y >>= 1;
18         x = matrix_muti(x, x, MOD);
19     }
20     return res;
21 }

```

5 D Data Structure

5.1 1D-BIT

```

1 int n; ll arr[N], pre[N], BIT[N];
2 // BIT[i] = pre[i] - pre[i - lowbit(i)]
3 void upd(ll x, ll dif) {
4     for(int i = x; i <= n; i += (i & -i))
5         BIT[i] += dif;
6 }
7 ll sum(ll x) { // prefix_sum 1~x
8     ll res = 0;
9     for(int i = x; i > 0; i -= (i & -i)) {
10        res = res + BIT[i];

```

```

11     }
12     return res;
13 }

```

5.2 2D-BIT

```

1 ll grid[N][N], BIT[N][N]; int n;
2 ll sum(int x, int y) { // sum of rectangle (1, 1) ~ (x, y)
3     ll res = 0;
4     for(int i = x; i > 0; i -= (i & -i)) {
5         for(int j = y; j > 0; j -= (j & -j))
6             res += BIT[i][j];
7     }
8     return res;
9 }
10 void upd(int x, int y, ll dif) { // update the value of (x, y)
11     by dif
12     for(int i = x; i <= n; i += (i & -i)) {
13         for(int j = y; j <= n; j += (j & -j))
14             BIT[i][j] += dif;
15     }

```

5.3 Mo's algorithm

```

1 int B = 500;
2 struct Range {
3     int L, R, i;
4     bool operator < (Range a) { return make_pair((L / B), R) <
5         make_pair(a.L / B, a.R); }
6 } query[N];
7 int l = 1, r = 0; // 1-index
8 for(int i = 0; i < q; i++) {
9     int ql = query[i].L, qr = query[i].R;
10    while(r < qr) add(++r);
11    while(r > qr) del(r--);
12    while(l < ql) del(l++);
13    while(l > ql) add(--l);
14    ans[query[i].i] = cur_cnt;

```

5.4 Sparse Table

```

1 void build_sparse_table() {
2     for(int i = 0; i < n; i++) sp[0][i] = arr[i];
3     for(int i = 1; (1 << i) <= n; i++) {
4         for(int j = 0; j + (1 << i) <= n; j++) {
5             sp[i][j] = min(sp[i - 1][j], sp[i - 1][j + (1 << (i
6                 - 1))]);
7         }
8     }
9 }
10 ll query(int left, int right) {
11     // or int_log2
12     int k = log2(right - left + 1);
13     return min(sp[k][left], sp[k][right - (1 << k) + 1]);

```

5.5 Treap

```

1 /* This is for Sum Query and Reverse Query
2  * Sum Query : Split root to Left, Mid, Right, ans = Mid->sum
3  * Reverse Query : Split root to Left, Mid, Right, and mark
4    swap_tag at Mid and Merge back into a root
5  * use srand(time(0) * clock()); */
6 struct Treap {
7     int size, pri, swap_tag;
8     ll val, sum;
9     Treap *lc, *rc;
10    Treap(ll num) {
11        swap_tag = 0, size = 1;
12        pri = rand();
13        val = sum = num;
14        lc = rc = NULL;
15    }
16 };
17 int size(Treap *node) {
18     if(node == NULL) return 0;
19     else return node->size;
20 }
21 ll sum(Treap *node) {
22     if(node == NULL) return 0;
23     else return node->sum;

```

```

24 void push(Treap *node) {
25     assert(node != NULL);
26     if(node != NULL) {
27         if(node->swap_tag) {
28             if(node->lc != NULL) node->lc->swap_tag ^= 1;
29             if(node->rc != NULL) node->rc->swap_tag ^= 1;
30             swap(node->lc, node->rc);
31         }
32         node->swap_tag = 0;
33     }
34 }
35 void split(Treap *node, Treap *&left, Treap *&right, int k) {
36     if(node == NULL) left = right = NULL;
37     else {
38         push(node);
39         if(size(node->lc) < k) {
40             split(node->rc, node->rc, right, k - size(node->lc)
41                 - 1);
42             left = node;
43         } else {
44             split(node->lc, left, node->lc, k);
45             right = node;
46         }
47         node->size = size(node->lc) + size(node->rc) + 1;
48         node->sum = sum(node->lc) + sum(node->rc) + node->val;
49     }
50 }
51 void merge(Treap *&node, Treap *left, Treap *right) {
52     if(left == NULL) node = right;
53     else if(right == NULL) node = left;
54     else {
55         if(left->pri < right->pri) {
56             push(left);
57             merge(left->rc, left->rc, right);
58             node = left;
59         } else {
60             push(right);
61             merge(right->lc, left, right->lc);
62             node = right;
63         }
64         node->size = size(node->lc) + size(node->rc) + 1;
65         node->sum = sum(node->lc) + sum(node->rc) + node->val;
66     }

```

5.6 Union-Find

```

1 int p[N], sz[N];
2 int find(int x) {
3     if(p[x] == x) return x;
4     return p[x] = find(p[x]);
5 }
6 void unite(int a, int b) {
7     a = find(a); b = find(b);
8     if(a == b) return;
9     if(sz[a] > sz[b]) swap(a, b);
10    p[a] = p[b];
11    sz[b] += sz[a];
12 }
13 void init() {
14     for(int i = 0; i < N; i++)
15         sz[i] = 1; p[i] = i;
16 }

```

6 DP optimization

6.1 Divide and Conquer

```

1 /* Divide into k subarrays
2  * The cost of each subarray is the square of the sum of the
3    values in the subarray
4  * Find the minimum sum of k costs of subarray */
5 /* dp[i][k] = minimum value of split pre i-th number into k
6    subarray */
7 void compute(int j, int left, int right, int left_bound, int
8     right_bound) {
9     if(left > right) return;
10    int mid = (left + right) / 2;
11    pair<ll, int> best; // val, pos
12    best = make_pair(INF, -1);
13    for(int i = max(j, left_bound); i <= right_bound; i++) {
14        /* cost[left][right] = the square of sum of (arr[left
15            ... right]) */
16        best = min(best, make_pair(dp[i - 1][j - 1] + cost[i][
17            mid], i));
18    }
19    dp[mid][j] = best.first;
20    compute(j, left, mid - 1, left_bound, best.second);
21    compute(j, mid + 1, right, best.second, right_bound);

```

```

17 }
18 int main() {
19     for(int i = 1; i <= n; i++) { dp[i][1] = cost[1][i]; }
20     for(int j = 2; j <= k; j++) { compute(j, 1, n, 1, n); }
21 }

```

6.2 DP Optimization Note

Divide and Conquer Optimization:

$solve(i, j) = \min(solve(p-1, j-1) + cost(p, i))$

condition:

$cost(a, c) + cost(b, d) \leq cost(a, d) + cost(b, c)$

then:

$pos(i, j) \leq pos(i+1, j)$

Knuth's Optimization

$solve(i, j) = \min(cost(i, j) + solve(i, p) + solve(p+1, j))$

condition:

$cost(b, c) \leq cost(a, d)$

and

$cost(a, c) + cost(b, d) \leq cost(a, d) + cost(b, c)$

then:

$pos(i, j-1) \leq pos(i, j) \leq pos(i+1, j)$

6.3 Knuth Optimization

```

1  /* Given an array of n numbers, your task is to divide it into
2     n number */
3  /* On one move, choose any subarray and split it into two
4     subarrays. */
5  /* The cost of such a move is the sum of values in the chosen
6     subarray. */
7  /* pos[left][right] = best split position (left < pos[left][
8     right] < right) */
9  /* dp[left][right] = minimum cost the split every number in (
10     left, right) */
11 for(int i = 1; i <= n; i++) { pos[i][i] = i; }
12 for(int l = 2; l <= n; l++) {
13     for(int i = 1; i + l - 1 <= n; i++) {
14         // compute dp[i][i + l - 1]
15         int left = i, right = i + l - 1;
16         pair<ll, int> best;
17         best = make_pair(INF, -1);
18         for(int p = pos[left][right - 1]; p <= pos[right + 1][
19             right]; p++)
20             best = min(best, make_pair(dp[left][p] + dp[p + 1][
21                 right] + cost(left, right), p));
22         pos[left][right] = best.second;
23         dp[left][right] = best.first;
24     }
25 }

```

6.4 LiChao

```

1 struct Line {
2     ll a, b; // ax + b
3     ll val(ll x) { return a * x + b; }
4     Line() {}
5     Line(ll _a, ll _b) : a(_a), b(_b) {}
6 };
7 struct Node {
8     int L, R;
9     Line line;
10    Node *lc, *rc;
11 };
12 void build(Node *r, int L, int R) {
13     r->L = L, r->R = R;
14     r->line = Line(INF, INF);
15     if(L == R) return;
16     int M = (L + R) / 2;
17     build(r->lc = new Node(), L, M);
18     build(r->rc = new Node(), M + 1, R);
19 }
20 void update(Node *r, Line line) {
21     Line left_line, right_line;
22     if(line.a > r->line.a) {
23         left_line = line;
24         right_line = r->line;
25     } else {
26         left_line = r->line;
27         right_line = line;
28     }
29     int M = (r->L + r->R) / 2;
30     if(left_line.val(M) < right_line.val(M)) {
31         r->line = left_line;
32         if(r->L != r->R) update(r->rc, right_line);
33     } else {

```

```

34         r->line = right_line;
35         if(r->L != r->R) update(r->lc, left_line);
36     }
37 }
38 ll qry(Node *r, int x) {
39     if(r->L == r->R) return r->line.val(x);
40     int M = (r->L + r->R) / 2;
41     if(x <= M) return min(r->line.val(x), qry(r->lc, x));
42     else return min(r->line.val(x), qry(r->rc, x));
43 }

```

7 E Tree algorithm

7.1 Cetroid Decomposition

```

1 // Fixed-Length Path II :
2 // to find the number of path k1 <= Length <= k2
3 vector<int> adj[N], node;
4 bool vis[N];
5 vector<pair<int, int>> dis_fa;
6 // dis_fa[i] = {length, subtree's root}
7 int sz[N], max_sz[N], cnt[N], k1, k2;
8 void DFS_dis(int x, int p, int sub_fa, int level) {
9     dis_fa.emplace_back(level, sub_fa);
10    for(auto v : adj[x]) {
11        if(v != p && !vis[v]) DFS_dis(v, x, sub_fa, level + 1);
12    }
13 }
14 void DFS_SZ(int x) {
15     sz[x] = 1; vis[x] = 1;
16     max_sz[x] = 0; node.push_back(x);
17     for(auto v : adj[x]) {
18         if(!vis[v]) {
19             DFS_SZ(v);
20             sz[x] += sz[v];
21             max_sz[x] = max(max_sz[x], sz[v]);
22         }
23     }
24 }
25 void init() { dis_fa.clear(); node.clear(); }
26 ll ans;
27 void DFS(int x) {
28     init();
29     DFS_SZ(x);
30     int n = (int)node.size(), centroid = -1;
31     for(auto v : node) {
32         vis[v] = 0; cnt[v] = 0;
33         int mx_comp = max(n - sz[v], max_sz[v]);
34         if(mx_comp * 2 <= n) centroid = v;
35     }
36     vis[centroid] = 1;
37     dis_fa.emplace_back(0, centroid);
38     for(auto v : adj[centroid]) {
39         if(!vis[v]) DFS_dis(v, centroid, v, 1);
40     }
41     sort(dis_fa.begin(), dis_fa.end());
42     int m = (int)dis_fa.size();
43     int left = m, right = m - 1;
44     // use Two Pointer
45     for(int i = 0; i < m; i++) {
46         int len = dis_fa[i].first;
47         int sub_fa = dis_fa[i].second;
48         while(right != -1 && dis_fa[right].first > k2 - len) {
49             cnt[dis_fa[right].second]--;
50             right--;
51         }
52         while(left && dis_fa[left - 1].first >= k1 - len) {
53             left--;
54             cnt[dis_fa[left].second]++;
55         }
56         ans += (right - left + 1);
57         ans -= (cnt[sub_fa]); // in Same Subtree
58     }
59     for(auto v : adj[centroid]) {
60         if(!vis[v]) DFS(v);
61     }
62 }
63 // DFS(s) to calculate ans

```

7.2 Change Root DP

```

1 // sum[i] = the sum of the distances from the node i to all
2 // other nodes
3 // cnt[i] = the sum of the distances from the node i to node in
4 // subtree
5 // subtree[i] = size of subtree
6 ll cnt[N], subtree[N], sum[N];
7 void toleaf(int x, int par) {
8     subtree[x] = 1;

```

```

7   for(auto i : adj[x]) {
8       if(i != par) {
9           toleaf(i, x);
10          subtree[x] += subtree[i];
11          cnt[x] += cnt[i] + subtree[i];
12      }
13  }
14  }
15  void dis_sum(int x, int par) {
16      if(par != 0) sum[x] = sum[par] - subtree[x] + (n - subtree[
17          x]);
18      else sum[x] = cnt[x];
19      for(auto i : adj[x]) if(i != par) dis_sum(i, x);
20  }

```

7.3 DSU on Tree

```

1  void DFS(int x, int par) {
2      subtree_color[x][color[x]]++;
3      for(auto v : adj[x]) {
4          if(v != par) {
5              DFS(v, x);
6              if(subtree_color[v].size() > subtree_color[x].size()
7                  ())
8                  swap(subtree_color[v], subtree_color[x]);
9              for(auto p : subtree_color[v]) { // merge
10                  subtree_color[x][p.first] += p.second;
11              }
12          }
13      }
14      ans[x] = (int)subtree_color[x].size(); // offline

```

7.4 Heavy-Light Decomposition

```

1  int sz[N], par[N], to[N], fr[N], dep[N];
2  int tree_arr[N], val[N], dfn[N];
3  void DFS(int x, int p) {
4      sz[x] = 1; par[x] = p;
5      dep[x] = dep[p] + 1; to[x] = -1;
6      for(auto v : adj[x]) {
7          if(v != p) {
8              DFS(v, x);
9              sz[x] += sz[v];
10             if(to[x] == -1 || sz[v] > sz[to[x]]) to[x] = v;
11         }
12     }
13 }
14 void Link(int x, int f) {
15     static int ord = 1;
16     dfn[x] = ord; fr[x] = f;
17     tree_arr[ord++] = x;
18     // and use Data Structure maintain tree_arr
19     if(to[x] != -1) Link(to[x], f);
20     for(int v : adj[x]) {
21         if(v == par[x] || v == to[x]) continue;
22         Link(v, v);
23     }
24 }
25 vector<pair<int, int>> Query(int u, int v) {
26     // return the intervals on the path from u to v
27     int fu = fr[u], fv = fr[v];
28     vector<pair<int, int>> res;
29     while(fu != fv) {
30         if(dep[fu] < dep[fv]) {
31             swap(fu, fv);
32             swap(u, v);
33         }
34         res.emplace_back(dfn[fu], dfn[u]);
35         u = par[fu];
36         fu = fr[u];
37     }
38     if(dep[u] > dep[v]) swap(u, v);
39     // u is LCA
40     res.emplace_back(dfn[u], dfn[v]);
41     return res;
42 }

```

7.5 LCA

```

1  int par[N][MAX_log], dep[N];
2  void DFS(int level, int x, int p) {
3      dep[x] = level;
4      for (auto i : adj[x]) {
5          if (i == p) continue;
6          par[i][0] = x;
7          DFS(level + 1, i, x);

```

```

8      }
9  }
10 void build() {
11     for (int i = 1; i < MAX_log; i++) {
12         for (int j = 1; j <= n; j++)
13             par[j][i] = par[par[j][i - 1]][i - 1];
14     }
15 }
16 int LCA(int a, int b) {
17     if(dep[a] > dep[b]) swap(a, b);
18     int dif = dep[b] - dep[a];
19     for(int i = 0; i < MAX_log; i++) {
20         if(dif & (1 << i))
21             b = par[b][i];
22     }
23     if(a == b) return a;
24     for(int i = MAX_log - 1; i >= 0; i--) { // must be form
25         MAX_Log ~ 0
26         if(par[a][i] != par[b][i]) {
27             a = par[a][i];
28             b = par[b][i];
29         }
30     }
31     return par[a][0];
32 }
33 void LCA_init() {
34     DFS(0, 1, 0);
35     build();
36 }
37 int Distance_Query(int a, int b) {
38     return dep[a] + dep[b] - 2 * dep[LCA(a,b)];

```

7.6 Tree Diameter

```

1  void DFS(int x, ll dis, ll &mx, ll p) {
2      dep[x] = dis;
3      if(dep[mx] <= dep[x]) mx = x;
4      for(auto v : adj[x]) {
5          if(v == p) continue;
6          DFS(v, dis + 1, mx, x);
7      }
8  }
9  ll Tree_Diameter() {
10     ll c = 1;
11     DFS(1, 0, c, -1);
12     DFS(c, 0, c, -1);
13     return dep[c];
14 }

```

7.7 Tree Traversals Array

```

1  void DFS(int x) { // use data structure to maintain tree_arr
2      vis[x] = 1; sz[x] = 1;
3      node_to_index[x] = (int)tree_arr.size();
4      tree_arr.push_back(x);
5      for(auto v : adj[x]) {
6          if(!vis[v]) {
7              DFS(v);
8              sz[x] += sz[v];
9          }
10     }
11 }

```

8 F String

8.1 Aho-Corasick

```

1  #include "Trie"
2  void GetAC() {
3      queue<int> que;
4      for(int i = 0; i < W; i++)
5          if(trie[0][i]) que.push(trie[0][i]);
6      while(!que.empty()) {
7          int now = que.front(); que.pop();
8          for(int i = 0; i < W; i++) {
9              int &to = trie[now][i], tmp;
10             if(to == 0) to = trie[fail[now]][i];
11             else {
12                 que.push(to), tmp = fail[now];
13                 while(tmp && trie[tmp][i] == 0) tmp = fail[tmp];
14                 fail[to] = trie[tmp][i];
15                 cnt[to] += cnt[trie[tmp][i]];
16             }

```

```

17     }
18 }
19 }
20 void Matching(string &s) {
21     int now = 0;
22     for(int i = 0; i < (int)s.size(); i++) {
23         while(now && trie[now][s[i] - 'a'] == 0)
24             now = fail[now];
25         if(trie[now][s[i] - 'a'])
26             now = trie[now][s[i] - 'a'];
27         if(cnt[now] > 0)
28             // find cnt[now] matches at i
29     }
30 }

```

8.2 KMP

```

1 int F[N], nextState[N][W];
2 void Failure(string &pat) { // build Failure Function
3     int n = (int)pat.size(), j = 0;
4     F[0] = 0;
5     for(int i = 1; i < n; i++) {
6         while(j != 0 && pat[i] != pat[j])
7             j = F[j - 1];
8         if(pat[j] == pat[i])
9             j++;
10        F[i] = j;
11    }
12    // --- Optional ---
13    for(int i = 0; i <= n; i++) {
14        for(int k = 0; k < W; k++) {
15            if(i < n && pat[i] == k + 'A') {
16                nextState[i][k] = i + 1;
17            } else if(i) nextState[i][k] = nextState[F[i - 1]][k];
18        }
19    }
20 }
21 int Matching(string &pat, string &text) {
22     int res = 0, j = 0, n = (int)text.size();
23     for(int i = 0; i < n; i++) {
24         while(j != 0 && pat[j] != text[i])
25             j = F[j - 1];
26         if(text[i] == pat[j]) {
27             if(j == (int)pat.size() - 1) {
28                 res++;
29                 j = F[j];
30             } else j++;
31         }
32     }
33     return res;
34 }

```

8.3 Manacher

```

1 int P[2 * N];
2 void Manacher(string &ss) {
3     string s;
4     s.resize(2 * (int)ss.size() + 1, '.');
5     for(int i = 0; i < (int)ss.size(); i++)
6         s[2 * i + 1] = ss[i];
7     // ss = abacabd, s = .a.b.a.c.a.b.d.
8     int l = 0, r = 0, n = (int)s.size();
9     for(int i = 0; i < n; i++) {
10        while(l + 1 - i < 0) l++;
11        P[i] = max(min(P[l + 1 - i], r - i), 1);
12        while(0 <= i - P[i] && i + P[i] < n && s[i - P[i]] == s[i + P[i]]) {
13            P[i]++;
14            l = i, r = i + P[i];
15        }
16    }
17    /* P[i] is the Radius of Longest Palindrome that center is
18       Position i
19       * s = . a . b . a . c . a . b . d .
20       * P = 1 2 1 4 1 2 1 6 1 2 1 2 1 2 1 */
21 }

```

8.4 Minimal Rotation

```

1 // rotate(s.begin(), s.begin() + minRotation(s), s.end());
2 int minRotation(string s) {
3     int a = 0, n = (int)s.size();
4     s = s + s;
5     for(int b = 0; b < n; b++) {
6         for(int k = 0; k < n; k++) {
7             if(a + k == b || s[a + k] < s[b + k]) {

```

```

8                 b += max(0, k - 1);
9                 break;
10            }
11            if(s[a + k] > s[b + k]) {
12                a = b;
13                break;
14            }
15        }
16    }
17    return a;
18 }

```

8.5 String Hash

```

1 #define MOD ((1ll)1e9 + 7) // 1004535809, 1e9 + 9
2 #define B (137LL) // 257LL, 311LL (311LL is best)
3 ll H[N], Pow_B[N];
4 void Rolling_Hash(string &s) {
5     int n = (int)s.size();
6     Pow_B[0] = 1;
7     for(int i = 1; i <= n; i++) { // H is 1-index
8         H[i] = (H[i - 1] * B + s[i - 1]) % MOD;
9         Pow_B[i] = Pow_B[i - 1] * B % MOD;
10    }
11 }
12 ll get_hash(int left, int right) {
13     return ((H[right + 1] - H[left] * Pow_B[right - left + 1]
14             % MOD) + MOD) % MOD;
15 }
16 ll all_hash(string &s) {
17     ll res = 0;
18     for(auto c : s)
19         res = (res * B + c) % MOD;
20     return res;
21 }

```

8.6 Suffix Array + LCP

```

1 int cnt[N], pos[N];
2 vector<pair<pair<int, int>, int>> a_new(N);
3 void radix_sort(vector<pair<pair<int, int>, int>> &a) {
4     int n = (int)a.size();
5     {
6         memset(cnt, 0, sizeof(cnt));
7         for(auto x : a) cnt[x.first.second]++;
8         pos[0] = 0;
9         for(int i = 1; i < n; i++) pos[i] = pos[i - 1] + cnt[i - 1];
10        for(auto x : a) {
11            int i = x.first.second;
12            a_new[pos[i]] = x;
13            pos[i]++;
14        }
15        for(int i = 0; i < n; i++) a[i] = a_new[i];
16    }
17    {
18        memset(cnt, 0, sizeof(cnt));
19        for(auto x : a) cnt[x.first.first]++;
20        pos[0] = 0;
21        for(int i = 1; i < n; i++) pos[i] = pos[i - 1] + cnt[i - 1];
22        for(auto x : a) {
23            int i = x.first.first;
24            a_new[pos[i]] = x;
25            pos[i]++;
26        }
27        for(int i = 0; i < n; i++) a[i] = a_new[i];
28    }
29 }
30 int sa[N], c[N];
31 void SA(string s) {
32     s += "$";
33     int n = (int)s.size();
34     {
35         vector<pair<pair<int, int>, int>> a(n);
36         for(int i = 0; i < n; i++) a[i] = {s[i], i};
37         sort(a.begin(), a.end());
38         for(int i = 0; i < n; i++) sa[i] = a[i].second;
39         c[sa[0]] = 0;
40         for(int i = 1; i < n; i++) {
41             if(a[i].first == a[i - 1].first) c[sa[i]] = c[sa[i - 1]];
42             else c[sa[i]] = c[sa[i - 1]] + 1;
43         }
44     }
45     for(int k = 0; (1 << k) <= n; k++) {
46         vector<pair<pair<int, int>, int>> a(n);
47         for(int i = 0; i < n; i++) a[i] = {(c[i], c[(i + (1 << k)) % n])}, i}; // % n
48         radix_sort(a);
49         for(int i = 0; i < n; i++) sa[i] = a[i].second;
50         c[sa[0]] = 0;
51         for(int i = 1; i < n; i++) {
52             if(a[i].first == a[i - 1].first) c[sa[i]] = c[sa[i - 1]];
53             else c[sa[i]] = c[sa[i - 1]] + 1;
54         }
55     }
56 }

```

```

49     }
50     // s.substr(sa[i], n - sa[i])
51 }
52 // sa[1~n] is the Suffix String Array
53 int lcp[N], rk[N];
54 void LCP(string s) {
55     s += '$';
56     int n = (int)s.size();
57     int pre = 0;
58     for(int i = 0; i < n; i++) rk[sa[i]] = i;
59     for(int i = 0; i < n; i++) {
60         if(rk[i] == 0) lcp[0] = 0;
61         else {
62             int j = sa[rk[i] - 1];
63             if(pre) pre--;
64             while(i + pre < n && j + pre < n && s[i + pre] == s[j + pre]) pre++;
65             lcp[rk[i]] = pre;
66         }
67     }
68 }
69 // Lcp[2~n]

```

8.7 Suffix Automaton

```

1 int state[N][W], link_to[N], length[N];
2 int last_state = 0, new_state = 1;
3 int clone(int x) {
4     int z = new_state++;
5     for(int i = 0; i < W; i++)
6         state[z][i] = state[x][i];
7     link_to[z] = link_to[x];
8     return z;
9 }
10 void SA(string &s) {
11     int n = (int)s.size();
12     link_to[0] = -1;
13     for(int i = 0; i < n; i++) {
14         int c = (s[i] - 'a');
15         int x = last_state;
16         int y = new_state++;
17         last_state = y;
18         length[y] = length[x] + 1;
19         link_to[y] = 0;
20         int s;
21         for(s = x; s != -1 && state[s][c] == 0; s = link_to[s])
22             state[s][c] = y;
23         if(s == -1) continue;
24         int u = state[s][c];
25         if(length[s] + 1 == length[u]) {
26             link_to[y] = u;
27             continue;
28         }
29         int z = clone(u);
30         length[z] = length[s] + 1;
31         for(int j = s; j != -1 && state[j][c] == u; j = link_to[j])
32             state[j][c] = z;
33         link_to[u] = link_to[y] = z;
34     }
35     for(int i = last_state; i != -1; i = link_to[i])
36         dp1[i] = 1;
37 }

```

8.8 Trie

```

1 int last_pos = 0, trie[N][W], cnt[N];
2 void Insert(string s) {
3     int now = 0;
4     for(auto c : s) {
5         int d = (c - 'a');
6         if(trie[now][d] == 0)
7             trie[now][d] = ++last_pos;
8         now = trie[now][d];
9     }
10    cnt[now]++;
11 }
12 void Print(int now, string &s) {
13     // s occurs cnt[now] times
14     for(int i = 0; i < W; i++) {
15         if(trie[now][i]) {
16             s += ('a' + i);
17             Print(trie[now][i], s);
18             s.pop_back();
19         }
20     }
21 }

```

8.9 Z algorithm

```

1 int Z[N];
2 void Z_algorithm(string &s) {
3     int n = (int)s.size();
4     int x = -1, y = -1;
5     Z[0] = 0;
6     for(int i = 1; i < n; i++) {
7         if(y < i) {
8             x = y = i;
9             while(y < n && s[y - x] == s[y])
10                 y++;
11             Z[i] = y - x;
12             y--;
13         } else if(Z[i - x] + i < y) {
14             Z[i] = Z[i - x];
15         } else {
16             x = i;
17             while(y < n && s[y - x] == s[y])
18                 y++;
19             Z[i] = y - x;
20             y--;
21         }
22     }
23 }

```

9 Flow

9.1 Max-Flow (Dinic)

```

1 struct Edge {
2     int to; ll cap; int rev;
3     Edge(){} // important
4     Edge(int _to, ll _cap, int _rev): to(_to), cap(_cap), rev(
5         _rev) {}
6 };
7 vector<Edge> adj[N];
8 int level[N], iter[N];
9 void add_edge(int from, int to, ll cap) {
10     adj[from].push_back(Edge(to, cap, (int)adj[to].size()));
11     adj[to].push_back(Edge(from, 0, (int)adj[from].size() - 1));
12 }
13 int s, t; // the max-flow of s to t;
14 void BFS() {
15     memset(level, -1, sizeof(level));
16     level[s] = 0;
17     queue<int> q;
18     q.push(s);
19     while(!q.empty()) {
20         int frt = q.front();
21         q.pop();
22         for(auto e : adj[frt]) {
23             if(e.cap > 0 && level[e.to] == -1) {
24                 level[e.to] = level[frt] + 1;
25                 q.push(e.to);
26             }
27         }
28     }
29 }
30 DFS(int now, ll flow) {
31     if(now == t) return flow;
32     for(int &i = iter[now]; i < (int)adj[now].size(); i++) {
33         // & is important for update iter[now]
34         Edge &e = adj[now][i];
35         if(e.cap > 0 && level[e.to] == level[now] + 1) {
36             ll ret = DFS(e.to, min(flow, e.cap));
37             if(ret > 0) {
38                 e.cap -= ret;
39                 adj[e.to][e.rev].cap += ret;
40                 return ret;
41             }
42         }
43     }
44     return 0;
45 }
46 ll max_flow() {
47     ll ret = 0;
48     while(1) {
49         BFS();
50         if(level[t] == -1) break;
51         memset(iter, 0, sizeof(iter));
52         ll tmp;
53         while((tmp = DFS(s, INF)) > 0) {
54             ret += tmp;
55         }
56     }
57     return ret;
58 }
59 int main() {
60     s = 1, t = n;

```

```

59 |     ans = max_flow();
60 | }

```

9.2 Min Cost Max-Flow

```

1 | struct Edge {
2 |     ll to, cap, cost, rev;
3 |     Edge() {}
4 |     Edge(int to, ll cap, ll cost, int rev) : to(to), cap(
5 |         _cap), cost(_cost), rev(_rev) {}
6 | };
7 | vector<Edge> adj[N];
8 | ll dis[N]; int par[N], par_id[N]; bool in_que[N];
9 | pair<ll, ll> MCMF(int s, int t) {
10 |     ll flow = 0, cost = 0;
11 |     while(true) {
12 |         memset(dis, 0x3f, sizeof(dis));
13 |         memset(in_que, 0, sizeof(in_que));
14 |         queue<int> que;
15 |         que.push(s); dis[s] = 0;
16 |         while(!que.empty()) { // SPFA
17 |             int x = que.front();
18 |             que.pop(); in_que[x] = 0;
19 |             for(int i = 0; i < (int)adj[x].size(); i++) {
20 |                 Edge &e = adj[x][i];
21 |                 if(e.cap > 0 && dis[e.to] > dis[x] + e.cost) {
22 |                     dis[e.to] = dis[x] + e.cost;
23 |                     par[e.to] = x;
24 |                     par_id[e.to] = i;
25 |                     if(!in_que[e.to]) {
26 |                         in_que[e.to] = 1;
27 |                         que.push(e.to);
28 |                     }
29 |                 }
30 |             }
31 |             if(dis[t] >= INF) break;
32 |             ll mi_flow = INF;
33 |             for(int i = t; i != s; i = par[i])
34 |                 mi_flow = min(mi_flow, adj[par[i]][par_id[i]].cap);
35 |             flow += mi_flow, cost += mi_flow * dis[t];
36 |             for(int i = t; i != s; i = par[i]) {
37 |                 Edge &e = adj[par[i]][par_id[i]];
38 |                 e.cap -= mi_flow;
39 |                 adj[e.to][e.rev].cap += mi_flow;
40 |             }
41 |         }
42 |         return make_pair(flow, cost);
43 |     }
44 | void add_edge(int a, int b, ll cap, ll cost) {
45 |     adj[a].push_back(Edge(b, cap, cost, (int)adj[b].size()));
46 |     adj[b].push_back(Edge(a, 0, -cost, (int)adj[a].size() - 1));
47 | }
48 | ll hungarian(vector<vector<ll>> cost) {
49 |     int n = (int)cost.size();
50 |     for(int i = 0; i < n; i++) {
51 |         for(int j = 0; j < n; j++) {
52 |             int L = i, T = j + n;
53 |             add_edge(L, T, 1, cost[i][j]);
54 |         }
55 |     }
56 |     int s = 2 * n, t = 2 * n + 1;
57 |     for(int i = 0; i < n; i++) {
58 |         int L = i, T = i + n;
59 |         add_edge(s, L, 1, 0);
60 |         add_edge(T, t, 1, 0);
61 |     }
62 |     // MCMF(s, t)
63 |     for(int i = 0; i < n; i++) {
64 |         for(Edge &e : adj[i]) {
65 |             if(e.to == s) continue;
66 |             int row = i, col = e.to - n;
67 |             if(e.cap == 0)
68 |                 // (row + 1) match (col + 1)
69 |             }
70 |         }
71 |     }
72 |     return MCMF(s, t).second;

```

10 G Geometric Computing

10.1 Convex Hull

```

1 | vector<Point> Half_Hull(vector<Point> &point) {
2 |     vector<Point> half_hull;
3 |     for(int i = 0; i < (int)point.size(); i++) {
4 |         int sz;

```

```

5 |         while((sz = (int)half_hull.size()) > 1 && cross(
6 |             half_hull[sz - 2], point[i], half_hull[sz - 1]) <
7 |             0)
8 |             half_hull.pop_back();
9 |         half_hull.push_back(point[i]);
10 |     }
11 |     return half_hull;
12 | }
13 | vector<Point> Convex_Hull(vector<Point> point) {
14 |     sort(point.begin(), point.end(), [](Point a, Point b) {
15 |         if(a.x != b.x) return a.x < b.x;
16 |         return a.y < b.y;
17 |     });
18 |     vector<Point> upper = Half_Hull(point);
19 |     reverse(point.begin(), point.end());
20 |     vector<Point> lower = Half_Hull(point);
21 |     // the convex hull order : upper[0 ~ upper.size() - 2] +
22 |     // lower[0 ~ lower.size() - 2]; (the upper[0] is equal
23 |     // with lower.back)
24 |     if(upper.size() >= 1LL) upper.pop_back();
25 |     if(lower.size() >= 1LL) lower.pop_back();
26 |     vector<Point> Convex(upper.begin(), upper.end());
27 |     Convex.insert(Convex.end(), lower.begin(), lower.end());
28 |     return Convex;
29 | }

```

10.2 Half-Plane Intersection

```

1 | struct Halfplane {
2 |     Point p, pq; double angle;
3 |     Halfplane() {}
4 |     Halfplane(const Point& a, const Point& b) : p(a), pq(b - a)
5 |     {
6 |         angle = atan2(pq.y, pq.x);
7 |     }
8 |     bool out(const Point& r) { return cross(pq, r - p) < -eps;
9 |     }
10 |     bool operator < (const Halfplane& e) const { return angle <
11 |         e.angle; }
12 |     friend Point inter(const Halfplane& s, const Halfplane& t)
13 |     {
14 |         double alpha = cross((t.p - s.p), t.pq) / cross(s.pq, t
15 |             .pq);
16 |         return s.p + (s.pq * alpha);
17 |     }
18 | };
19 | vector<Point> hp_intersect(vector<Halfplane>& H) {
20 |     Point box[4] = {
21 |         Point(-inf, inf),
22 |         Point(inf, inf),
23 |         Point(inf, -inf),
24 |         Point(-inf, -inf)
25 |     };
26 |     for(int i = 0; i < 4; i++) {
27 |         Halfplane aux(box[i], box[(i+1) % 4]);
28 |         H.push_back(aux);
29 |     }
30 |     sort(H.begin(), H.end());
31 |     deque<Halfplane> dq;
32 |     int len = 0;
33 |     for(int i = 0; i < (int)H.size(); i++) {
34 |         while (len > 1 && H[i].out(inter(dq[len-1], dq[len-2]))
35 |             )
36 |             dq.pop_back(), --len;
37 |         while (len > 1 && H[i].out(inter(dq[0], dq[1])))
38 |             dq.pop_front(), --len;
39 |         if (len > 0 && fabs1(cross(H[i].pq, dq[len-1].pq)) <
40 |             eps) {
41 |             if (dot(H[i].pq, dq[len-1].pq) < 0.0)
42 |                 return vector<Point>();
43 |             if (H[i].out(dq[len-1].p)) {
44 |                 dq.pop_back(), --len;
45 |             } else continue;
46 |         }
47 |         dq.push_back(H[i]), ++len;
48 |     }
49 |     while (len > 2 && dq[0].out(inter(dq[len-1], dq[len-2])))
50 |         dq.pop_back(), --len;
51 |     while (len > 2 && dq[len-1].out(inter(dq[0], dq[1])))
52 |         dq.pop_front(), --len;
53 |     if (len < 3) return vector<Point>();
54 |     vector<Point> ret(len);
55 |     for(int i = 0; i+1 < len; i++)
56 |         ret[i] = inter(dq[i], dq[i+1]);
57 |     ret.back() = inter(dq[len-1], dq[0]);
58 |     return ret;
59 | }

```

10.3 Heart

```

1 pdd circenter(pdd p0, pdd p1, pdd p2) { // radius = abs(center)
2   p1 = p1 - p0, p2 = p2 - p0;
3   double x1 = p1.X, y1 = p1.Y, x2 = p2.X, y2 = p2.Y;
4   double m = 2. * (x1 * y2 - y1 * x2);
5   center.X = (x1 * x1 * y2 - x2 * x2 * y1 + y1 * y2 * (y1 -
6     y2)) / m;
7   center.Y = (x1 * x2 * (x2 - x1) - y1 * y1 * x2 + x1 * y2 *
8     y2) / m;
9   return center + p0;
10 }
11 pdd incenter(pdd p1, pdd p2, pdd p3) { // radius = area / s * 2
12   double a = abs(p2 - p3), b = abs(p1 - p3), c = abs(p1 - p2);
13   ;
14   double s = a + b + c;
15   return (a * p1 + b * p2 + c * p3) / s;
16 }
17 pdd masscenter(pdd p1, pdd p2, pdd p3)
18 { return (p1 + p2 + p3) / 3; }
19 pdd orthcenter(pdd p1, pdd p2, pdd p3)
20 { return masscenter(p1, p2, p3) * 3 - circenter(p1, p2, p3) *
21   2; }

```

10.4 Line Segment Intersection

```

1 const int LEFT = 1, RIGHT = 2, ON_LINE = 3;
2 int PointTest(Point a, Point b, Point p) {
3   ll res = cross(a, b, p);
4   if(res > 0) return LEFT;
5   if(res < 0) return RIGHT;
6   if(res == 0) return ON_LINE;
7 }
8 bool Intersect(Point a, Point b, Point p) {
9   bool res1 = (a.x <= p.x && p.x <= b.x) || (b.x <= p.x && p.
10     x <= a.x);
11   bool res2 = (a.y <= p.y && p.y <= b.y) || (b.y <= p.y && p.
12     y <= a.y);
13   return res1 && res2;
14 }
15 bool IsSegIntersect(Point a, Point b, Point p, Point q) {
16   int Tp = PointTest(a, b, p), Tq = PointTest(a, b, q);
17   int Ta = PointTest(p, q, a), Tb = PointTest(p, q, b);
18   bool Ip = Intersect(a, b, p), Iq = Intersect(a, b, q);
19   bool Ia = Intersect(p, q, a), Ib = Intersect(p, q, b);
20   /* case for on line */
21   if(Tp == ON_LINE && Ip) return true;
22   if(Tq == ON_LINE && Iq) return true;
23   if(Ta == ON_LINE && Ia) return true;
24   if(Tb == ON_LINE && Ib) return true;
25   /* case for not on line */
26   return (Tp != Tq) && (Ta != Tb);
27 }

```

10.5 Minimum Enclosing Circle

```

1 Point excenter(Point a, Point b, Point c, double &r) {
2   double a2 = a.x * a.x + a.y * a.y;
3   double b2 = b.x * b.x + b.y * b.y;
4   double c2 = c.x * c.x + c.y * c.y;
5   double d = 2 * (a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x
6     * (a.y - b.y));
7   double cx = (a2 * (b.y - c.y) + b2 * (c.y - a.y) + c2 * (a.
8     y - b.y)) / d;
9   double cy = (a2 * (c.x - b.x) + b2 * (a.x - c.x) + c2 * (b.
10     x - a.x)) / d;
11   Point center(cx, cy);
12   r = abs(center - a);
13   return center;
14 }
15 Point Minimum_Enclosing_Circle(vector<Point> pts, double &r) {
16   Point cent;
17   random_shuffle(pts.begin(), pts.end());
18   cent = pts[0], r = 0;
19   for (int i = 1; i < pts.size(); ++i)
20     if (abs(pts[i] - cent) > r) {
21       cent = pts[i], r = 0;
22       for (int j = 0; j < i; ++j)
23         if (abs(pts[j] - cent) > r) {
24           cent = (pts[i] + pts[j]) / 2;
25           r = abs(pts[i] - cent);
26           for (int k = 0; k < j; ++k)
27             if (abs(pts[k] - cent) > r)
28               cent = excenter(pts[i], pts[j], pts
29                 [k], r);
30         }
31     }
32   return cent;
33 }

```

10.6 Minimum Euclidean Distance

```

1 #define X second // important
2 #define Y first // important
3 typedef pair<ll, ll> Point;
4 ll dis(Point a, Point b) {
5   return (a.X - b.X) * (a.X - b.X) + (a.Y - b.Y) * (a.Y - b.Y)
6   );
7 }
8 ll Minimum_Euclidean_Distance(vector<Point> point) { // The
9   point arr and the size
10   int n = (int)point.size();
11   sort(point.begin(), point.end(), [](Point a, Point b) {
12     if(a.X != b.X) return a.X < b.X;
13     return a.Y < b.Y;
14   });
15   ll ans = (ll)8e18;
16   int left = -1;
17   set<Point> s;
18   for(int right = 0; right < n; right++) {
19     ll d = (ll)floor(sqrt(ans));
20     while(left + 1 < right && point[left + 1].X < point[
21       right].X - d) {
22       auto it = s.find(point[left + 1]);
23       assert(it != s.end()); s.erase(it);
24       left++;
25     }
26     // make_pair : First is Y, second is X
27     auto lb = s.lower_bound(make_pair(point[right].Y - d,
28       point[right].X - d));
29     auto ub = s.upper_bound(make_pair(point[right].Y + d,
30       point[right].X + d));
31     for(auto P = lb; P != ub; P++) {
32       ans = min(ans, dis(*P, point[right]));
33     }
34     s.insert(point[right]);
35   }
36   // distance = sqrt(ans), According to Problem
37   return ans;
38 }

```

10.7 Pick's Therom

The area of polygon = $a + b/2 - 1$
 a = number of lattices inside the polygon
 b = number of lattices on the boundary of polygon

10.8 Point and Cross

```

1 // ll or double
2 struct Point {
3   ll x, y;
4   int i;
5   Point(ll _x, ll _y, int _i = 0) : x(_x), y(_y), i(_i) {}
6   friend Point operator +(const Point& p, const Point& q) {
7     return Point(p.x + q.x, p.y + q.y);
8   }
9   friend Point operator - (const Point& p, const Point& q) {
10    return Point(p.x - q.x, p.y - q.y);
11  }
12  friend Point operator * (const Point& p, const ll& k) {
13    return Point(p.x * k, p.y * k);
14  }
15  friend ll dot(const Point& p, const Point& q) {
16    return p.x * q.x + p.y * q.y;
17  }
18  friend ll cross(const Point& p, const Point& q) {
19    return p.x * q.y - p.y * q.x;
20  }
21  friend double abs(Point a) { return sqrt(dot(a, a)); }
22 };
23 ll cross(Point a, Point b, Point c) { return cross(b - a, c - a
24   ); }
25 // two lines f(x) = m * x + c
26 // the intersection point
27 // x = (c2 - c1) / (m1 - m2)
28 // y = (m2c1 - m1c2) / (m2 - m1)

```

10.9 Points in Convex Hull

```

1 bool isInsideConvexPolygon(vector<Point>& hull, Point& p){
2   int n = hull.size();
3   if(n == 2) {
4     ll ux = max(hull[0].x, hull[1].x), lx = min(hull[0].x,
5       hull[1].x);
6     ll uy = max(hull[0].y, hull[1].y), ly = min(hull[0].y,
7       hull[1].y);
8     return (cross(p, hull[0], hull[1]) == 0 && lx <= p.x &&
9       p.x <= ux && ly <= p.y && p.y <= uy);
10  }
11  if(cross(hull[0], hull[1], p) < 0 || cross(hull[0], hull[n
12    -1], p) > 0) return false;
13  int l = 1, r = n - 1;

```

```

10 while(l + 1 < r){
11     int m = (l + r) / 2;
12     if(cross(hull[0], hull[m], p) >= 0) l = m;
13     else r = m;
14 }
15 return cross(hull[l], hull[r], p) >= 0;
16 }

```

10.10 PointSegDist

```

1 double PointSegDist(Point &p1, Point &p2, Point &p) {
2     ll dx = p1.x - p2.x, dy = p1.y - p2.y;
3     Point pr;
4     pr.x = p.x + dy;
5     pr.y = p.y - dx;
6     bool res1 = (cross(p, pr, p1) >= 0);
7     bool res2 = (cross(p, pr, p2) >= 0);
8     if(res1 == res2) return min(len(p, p1), len(p, p2));
9     else return fabs(cross(p, p1, p2)) / len(p1, p2);
10 }

```

10.11 Polygon Area

```

1 ll Polygon_Area(vector<Point> &point) {
2     ll ans = 0; int n = (int)point.size();
3     for(int i = 0; i < n; i++) {
4         Point p1 = {point[i].x, point[i].y};
5         Point p2 = {point[(i + 1) % n].x, point[(i + 1) % n].y};
6         ans += cross(p1, p2);
7     }
8     return ans > 0 ? ans : -ans;
9 }

```

11 H Number Theorem

11.1 Chinese Remainder Theorem

```

1 ll inv(ll n, ll p) {
2     ll x, y, z;
3     tie(x, y, z) = exgcd(n, p);
4     return (x % p + p) % p;
5 }
6 // ans % eqs[i].second = eqs[i].first
7 ll CRT(vector<pair<ll, ll>> eqs) {
8     ll M = 1, ans = 0, n = eqs.size();
9     for(int i = 0; i < n; i++)
10         M = M * eqs[i].second;
11     for(int i = 0; i < n; i++) {
12         ll Mi = M / eqs[i].second;
13         ans += eqs[i].first * inv(Mi, eqs[i].second) % M * Mi;
14     }
15     ans %= M;
16     return ans;
17 }

```

11.2 Coordinate Compression

```

1 vector<int> id_to_cor; map<ll, ll> compress;
2 void Coordinate_Compression() {
3     for(auto &x : compress) {
4         x.second = id_to_cor.size();
5         id_to_cor.push_back(x.first);
6     }
7 }

```

11.3 Determinant and Gaussian Elimination

```

1 // Use Gaussian Elimination
2 ll Determinant(vector<vector<ll>> &matrix) {
3     int n = matrix.size();
4     int m = matrix[0].size();
5     int row = 0;
6     ll ans = 1;
7     for(int col = 0; col < m; col++) {
8         ll nonzero_row = -1;
9         for(int i = row; i < n; i++) {
10             if(matrix[i][col]) {
11                 nonzero_row = i;

```

```

12                 break;
13             }
14         }
15         if(nonezero_row == -1) continue;
16         if(row != nonezero_row) ans *= -1;
17         swap(matrix[row], matrix[nonezero_row]);
18         for(int i = row + 1; i < n; i++) {
19             if(matrix[i][col] == 0) continue;
20             ll lm = lcm(matrix[row][col], matrix[i][col]);
21             ll frac1 = lm * inv(matrix[row][col]) % MOD;
22             ll frac2 = lm * inv(matrix[i][col]) % MOD;
23             for(int j = 0; j < m; j++) {
24                 matrix[i][j] *= frac2;
25                 matrix[i][j] -= (matrix[row][j] * frac1 % MOD);
26                 matrix[i][j] = (matrix[i][j] % MOD + MOD) % MOD;
27             }
28             ans *= inv(frac2);
29             ans %= MOD;
30         }
31         row++;
32     }
33     for(int i = 0; i < n; i++) {
34         ans *= matrix[i][i];
35         ans %= MOD;
36     }
37     return (ans % MOD + MOD) % MOD;
38     for(int row = n - 1; row >= 0; row--) {
39         for(int col = 0; col < m - 1; col++) {
40             if(matrix[row][col] == 1) {
41                 for(int i = row - 1; i >= 0; i--) {
42                     ll mul = matrix[i][col];
43                     for(int j = 0; j < m; j++) {
44                         matrix[i][j] -= matrix[row][j] * mul;
45                         matrix[i][j] = (matrix[i][j] % MOD + MOD) % MOD;
46                     }
47                 }
48                 break;
49             }
50         }
51     }
52 }

```

11.4 Extended Euclidean

```

1 tuple<ll, ll, ll> exgcd(ll a, ll b) { // x * a + y * b = gcd(a,
2     b)
3     if(b == 0) return make_tuple(1, 0, a);
4     ll x, y, z;
5     tie(x, y, z) = exgcd(b, a % b);
6     return make_tuple(y, x - (a / b) * y, z);
7 }
8 /* to calculate a / b = c (x % MOD) if gcd(b, MOD) = 1 (a and b
9    can mod MOD previously)
10 * tie(x, y, z) = exgcd(b, MOD); x * b + y * MOD = x * b = gcd(
11    b, MOD) = 1 => x = b^(-1)

```

11.5 Fast Power and BigInt Multiplication

```

1 ll fp(ll x, ll y, ll p) {
2     ll res = 1;
3     while(y) {
4         if(y & 1) res = res * x % p;
5         y >>= 1;
6         x = x * x % p;
7     }
8     return res;
9 }
10 ll muti(ll x, ll y, ll p) { // x * y % p
11     ll res = 0;
12     while(y) {
13         if(y & 1) res = (res + x) % p;
14         y >>= 1;
15         x = x * 2 % p;
16     }
17     return res;
18 }

```

11.6 FFT

```

1 typedef complex<double> cd;
2 const double PI = acos(-1);
3 vector<cd> FFT(vector<cd> &F, bool inv) {
4     int n = (int)F.size();
5     vector<cd> res(n);
6     for(int k = 0; k < n; k++) {

```

```

7   int RevBit = 0;
8   for(int i = 1; i < n; i <= 1) {
9       RevBit <= 1;
10      if(k & i) RevBit++;
11  }
12  res[RevBit] = F[k];
13  }
14  for(int Pow = 2; Pow <= n; Pow *= 2) {
15      double theta = (inv ? -1 : 1) * 2.0 * PI / Pow;
16      cd omega(cos(theta), sin(theta));
17      for(int rnd = 0; rnd < n; rnd += Pow) {
18          cd cur = 1;
19          for(int i = 0; i < Pow / 2; i++) {
20              cd a = res[rnd + i];
21              cd b = cur * res[rnd + i + Pow / 2];
22              res[rnd + i] = a + b;
23              res[rnd + i + Pow / 2] = a - b;
24              cur *= omega;
25          }
26      }
27  }
28  if(inv) { for(int i = 0; i < n; i++) res[i] /= n; }
29  return res;
30 }
31 vector<cd> Poly_multi(vector<cd> A, vector<cd> B) {
32     int k = (1 << ((int)log2((int)A.size() + (int)B.size()) +
33         1));
34     A.resize(k); B.resize(k);
35     A = FFT(A, 0); B = FFT(B, 0);
36     vector<cd> C(k);
37     for(int i = 0; i < k; i++) C[i] = A[i] * B[i];
38     C = FFT(C, 1);
39     return C;
40 }

```

11.7 Josephus

$$g(1, k) = 0$$

$$g(n, k) = (g(n - 1, k) + k) \% n$$

11.8 Nim Game

```

1 if(xor_sum) // first_player wins;
2 else // second_player wins;

```

12 MST

12.1 Kruskal

```

1 // #include Union-find
2 ll Kruskal() {
3     init(); // Union-Find init
4     // edge[i] = {a, b, w}
5     sort(edge, edge + m, [](Edge a, Edge b) { return a.w < b.w;
6     });
7     ll ans = 0; int cnt = 0;
8     for(int i = 0; i < m; i++) {
9         if(find(edge[i].a) == find(edge[i].b)) continue;
10        unite(edge[i].a, edge[i].b);
11        ans = ans + edge[i].w, cnt++;
12    }
13    return (cnt == n - 1) ? (ans) : (-1);
14 }

```

12.2 Prim

```

1 priority_queue<pii, vector<pii>, greater<pii>> pq; // (weight,
2     node)
3 vector<pii> adj[N]; // (neighbor, weight)
4 bool mark[N];
5 ll Prim() {
6     pq.push(pii(0, 1)); ll ans = 0;
7     while(!pq.empty()) {
8         int x = pq.top().second;
9         int weight = pq.top().first;
10        pq.pop();
11        if(mark[x]) continue;
12        mark[x] = true; ans += weight;
13        for(auto [v, w] : adj[x])
14            if(!mark[v]) pq.push(pii(w, v));
15    }
16 }

```

```

15 for(int i = 1; i <= n; i++)
16     if(!mark[i]) return 0;
17 return ans;
18 }

```

13 Prime and Composite

13.1 Factorize and Phi

```

1 vector<pair<ll, ll>> factorize(ll n) {
2     vector<pair<ll, ll>> fac;
3     for(ll i = 2; i * i <= n; i++) {
4         if(n % i == 0) fac.push_back({i, 0});
5         while(n % i == 0) {
6             fac.back().second++;
7             n = n / i;
8         }
9     }
10    if(n > 1) fac.push_back({n, 1});
11    return fac;
12 }
13 // the number of i, i <= n and gcd(n, i) = 1
14 ll phi(ll n, vector<pair<ll, ll>> fac) {
15     factorize(n, fac);
16     ll ans = 1;
17     for(auto [prime, power] : fac)
18         ans = ans * (prime - 1) * fp(prime, power - 1);
19     return ans;
20 }

```

13.2 Miller Rabin

```

1 // include Fast Power and BigInt Multiplication
2 // remember use the muti of BigInt in fp
3 void cal_rd(ll n, ll &r, ll &d) {
4     r = 0;
5     while(n % 2 == 0) {
6         r++;
7         n = n / 2;
8     }
9     d = n;
10 }
11 bool is_prime(ll n) {
12     if(n <= 1) return 0;
13     if(n == 2 || n == 3) return 1;
14
15     int times = 10;
16     ll r, d;
17     cal_rd(n - 1, r, d);
18     while(times--) {
19         ll a = unif(gen) % (n - 3) + 2;
20         ll x = fp(a, d, n);
21         if(x == 1 || x == n - 1)
22             continue;
23         bool probaly_prime = 0;
24         for(int i = 0; i < r - 1; i++) {
25             x = muti(x, x, n);
26             if(x == n - 1) {
27                 probaly_prime = 1;
28                 break;
29             }
30         }
31         if(probaly_prime)
32             continue;
33         return 0;
34     }
35     return 1;
36 }

```

13.3 Mobius

```

1 int mobius[N];
2 int LPF[N], cnt[N];
3 void Least_Prime_Factor() {
4     for(int i = 2; i <= N; i++) {
5         if(!LPF[i]) {
6             for(int j = i; j <= N; j += i) {
7                 if(!LPF[j]) LPF[j] = i;
8             }
9         }
10    }
11 }
12 void Mobius() {
13     Least_Prime_Factor();
14     mobius[1] = 1;
15 }

```

```

15 for(int i = 2; i <= N; i++) {
16     if(LPF[i / LPF[i]] == LPF[i]) {
17         mobius[i] = 0;
18     } else {
19         mobius[i] = -1 * mobius[i / LPF[i]];
20     }
21 }
22 }

```

13.4 Pollord rho

```

1 // include Simple Big Integer multiplication
2 ll g(ll x, ll rnd, ll p) {
3     return (rnd * muti(x, x, p) + 1) % p;
4 }
5 map<ll, int> fac;
6 void pollard_rho(ll n) {
7     if(!is_prime(n) && n > 1) {
8         ll d;
9         while(1) {
10             ll x = unif(gen) % n;
11             ll rnd = unif(gen) % 1000;
12             ll y = g(x, rnd, n);
13             d = 1;
14             while(d == 1 && x != y) {
15                 x = g(x, rnd, n);
16                 y = g(g(y, rnd, n), rnd, n);
17                 d = gcd(abs(x - y), n);
18             }
19             if(d == n) continue;
20             break;
21         }
22         pollard_rho(d);
23         pollard_rho(n / d);
24     } else if(n > 1)
25         fac[n]++;
26 }

```

13.5 Prime Table

```

1 const ll N = 1e6; // must be Composite
2 bool is_prime[N];
3 vector<ll> primes;
4 void init_primes() {
5     fill(is_prime, is_prime + N, true);
6     is_prime[0] = is_prime[1] = false;
7     for(ll i = 2; i < N; i++) {
8         if(is_prime[i]) {
9             primes.push_back(i);
10            for (ll j = i * i; j < N; j += i)
11                is_prime[j] = false;
12        }
13    }
14 }

```

14 SCC

14.1 2-SAT

n Logic Statements: $L_1, L_2, L_3, L_4 \dots L_n$
a 2-SAT Problem: $(x_1 \vee x_2) \wedge (x_3 \vee x_4) \dots (x_i \vee x_j)$
 x_i is L_k or $\neg L_k$
Solution: Build an implication Graph
For every clause $(x_i \vee x_j)$
Building two Edge $\neg x_i \rightarrow x_j, \neg x_j \rightarrow x_i$
And use Kosaraju to find SCC
This 2-SAT is unsatisfiable IFF
exist a statement $SCC_id[L_i] == SCC_id[\neg L_i]$
If the 2-SAT is satisfiable, for every statement,
True or False is chosen by Smaller SCC_id
Formally, if $SCC_id[L_i] < SCC_id[\neg L_i]$, L_i is True
Vice versa, if $SCC_id[L_i] > SCC_id[\neg L_i]$, L_i is False

14.2 Kosaraju

```

1 vector<int> adj[N], Revadj[N], ord;
2 bool vis[N]; int SCC_id[N];
3 int n, m;
4 void RevDFS(int x) {
5     vis[x] = 1;
6     for(auto v : Revadj[x]) {
7         if(!vis[v]) RevDFS(v);
8     }
9     ord.push_back(x);

```

```

10 }
11 void DFS(int x, int id) {
12     SCC_id[x] = id;
13     for(auto v : adj[x]) {
14         if(!SCC_id[v]) DFS(v, id);
15     }
16 }
17 int scc_id;
18 void Kosaraju() {
19     for(int i = 1; i <= n; i++) {
20         if(!vis[i]) RevDFS(i);
21     }
22     scc_id = 0;
23     for(int i = n - 1; i >= 0; i--) {
24         int x = ord[i];
25         if(!SCC_id[x]) DFS(x, ++scc_id);
26     }
27 }
28 void add_edge(int a, int b) {
29     adj[a].push_back(b);
30     Revadj[b].push_back(a);
31 }

```

14.3 Tarjan

```

1 int low[N], dfn[N], dfn_to_idx[N];
2 int SCC_cnt, SCC_id[N], n, m;
3 bool in_s[N];
4 vector<int> adj[N], s; // s is stack
5 void DFS(int x) {
6     static int ord = 1;
7     dfn[x] = low[x] = ord++;
8     s.push_back(x);
9     in_s[x] = 1;
10    for(auto v : adj[x]) {
11        if(!dfn[v]) {
12            DFS(v);
13            low[x] = min(low[x], low[v]);
14        }
15        else if(in_s[v])
16            low[x] = min(low[x], dfn[v]);
17    }
18    if(dfn[x] == low[x]) {
19        SCC_cnt++;
20        while(s.back() != x) {
21            SCC_id[s.back()] = SCC_cnt;
22            in_s[s.back()] = 0;
23            s.pop_back();
24        }
25        SCC_id[x] = SCC_cnt;
26        in_s[x] = 0;
27        s.pop_back();
28    }
29 }
30 void Tarjan_SCC() {
31     SCC_cnt = 0;
32     for(int i = 1; i <= n; i++)
33         if(!dfn[i]) DFS(i);
34 }

```

15 Segment Tree

15.1 Area of Union

```

1 struct Event {
2     ll x; ll y1, y2;
3     string type;
4     Event(ll _x, ll _y1, ll _y2, string _type): x(_x), y1(_y1),
5         y2(_y2), type(_type) {}
6     bool operator < (const Event a) {
7         return x < a.x;
8     }
9 };
10 map<ll, ll> compress;
11 vector<ll> id_to_cor;
12 vector<Event> SweepLine;
13 struct Node {
14     int L, R; ll uni, num;
15     Node *lc, *rc;
16     void pull() {
17         if(num) uni = id_to_cor[R] - id_to_cor[L];
18         else if(L != R - 1) uni = lc->uni + rc->uni;
19         else uni = 0;
20     }
21 };
22 void build(Node *r, int L, int R) {
23     r->L = L, r->R = R; r->uni = 0;
24     if(L == R - 1) return;
25     int M = (L + R) / 2;

```

```

25     build(r->lc = new Node(), L, M);
26     build(r->rc = new Node(), M, R);
27 }
28 ll qry_uni(Node *r) { return r->uni; }
29 void upd(Node *r, int ql, int qr, ll dif) {
30     if(ql >= r->R || qr <= r->L) return;
31     if(ql <= r->L && r->R <= qr) {
32         r->num += dif;
33         r->pull();
34         return;
35     }
36     upd(r->lc, ql, qr, dif);
37     upd(r->rc, ql, qr, dif);
38     r->pull();
39 }
40 struct Rectangle {
41     ll x1, x2, y1, y2;
42 };
43 ll Area_of_Union(vector<Rectangle> v) {
44     for(auto Rec : v) {
45         ll x1 = Rec.x1, x2 = Rec.x2;
46         ll y1 = Rec.y1, y2 = Rec.y2;
47         compress[y1], compress[y2];
48         SweepLine.push_back(Event(x1, y1, y2, "Add"));
49         SweepLine.push_back(Event(x2, y1, y2, "Del"));
50     }
51     Coordinate_Compression();
52     Node *root = new Node();
53     build(root, 0, (int)compress.size());
54     sort(SweepLine.begin(), SweepLine.end());
55     ll ans = 0, last_cor = SweepLine[0].x;
56     for(Event i : SweepLine) {
57         ans += qry_uni(root) * (i.x - last_cor);
58         if(i.type == "Add") {
59             upd(root, compress[i.y1], compress[i.y2], 1);
60         } else if(i.type == "Del") {
61             upd(root, compress[i.y1], compress[i.y2], -1);
62         }
63         last_cor = i.x;
64     }
65     return ans;
66 }

```

15.2 Lazy Segment Tree

```

1 // RMQ and range update
2 struct Node {
3     int L, R;
4     ll val, lazy_val;
5     Node *lc, *rc;
6     void pull() {
7         val = min(lc->val, rc->val);
8     }
9 };
10 void push(Node *r) {
11     if(!r->lazy_val) return;
12     if(r->L != r->R) {
13         r->lc->val += r->lazy_val;
14         r->rc->val += r->lazy_val;
15         r->lc->lazy_val += r->lazy_val;
16         r->rc->lazy_val += r->lazy_val;
17     }
18     r->lazy_val = 0;
19 }
20 ll arr[N];
21 void build(Node *r, int L, int R) {
22     r->L = L; r->R = R;
23     int M = (L + R) / 2;
24     r->lazy_val = 0;
25     if(L == R) {
26         r->val = arr[L];
27         return;
28     }
29     build(r->lc = new Node(), L, M);
30     build(r->rc = new Node(), M + 1, R);
31     r->pull();
32 }
33 void upd(Node *r, int ql, int qr, ll val) {
34     if(ql > r->R || qr < r->L) return;
35     if(ql <= r->L && r->R <= qr) {
36         r->val += val;
37         r->lazy_val += val;
38         return;
39     }
40     push(r);
41     upd(r->lc, ql, qr, val);
42     upd(r->rc, ql, qr, val);
43     r->pull(); // important
44 }
45 ll qry(Node *r, int ql, int qr) {
46     if(ql > r->R || qr < r->L) return INF;
47     if(ql <= r->L && r->R <= qr) return r->val;
48     push(r);
49     return min(qry(r->lc, ql, qr), qry(r->rc, ql, qr));

```

```
50 }
```

15.3 Persistent Segment Tree

```

1 ll arr[N];
2 struct Node {
3     int L, R;
4     ll sum;
5     Node *lc, *rc;
6     Node() { L = R = sum = 0, lc = rc = NULL; }
7     void pull() { sum = lc->sum + rc->sum; }
8 } *root_node[N]; // Store the history root
9 /* Use new_root = copy_node(old_root)
10  * and Update new_root */
11 Node *copy_node(Node *r) {
12     Node* cp = new Node();
13     cp->L = r->L; cp->R = r->R;
14     cp->sum = r->sum;
15     cp->lc = r->lc; cp->rc = r->rc;
16     return cp;
17 }
18 void build(Node *r, int L, int R) {
19     r->L = L; r->R = R;
20     if(L == R) {
21         r->sum = arr[L];
22         return;
23     }
24     int M = (L + R) / 2;
25     build(r->lc = new Node(), L, M);
26     build(r->rc = new Node(), M + 1, R);
27     r->pull();
28 }
29 void upd(Node *r, Node *new_r, int pos, ll val) {
30     if(r->L == r->R) {
31         new_r->sum = val;
32         return;
33     }
34     int M = (r->L + r->R) / 2;
35     if(pos <= M) {
36         new_r->lc = copy_node(r->lc);
37         upd(r->lc, new_r->lc, pos, val);
38     }
39     else {
40         new_r->rc = copy_node(r->rc);
41         upd(r->rc, new_r->rc, pos, val);
42     }
43     new_r->pull();
44 }
45 ll qry(Node *r, int ql, int qr) {
46     if(ql > r->R || qr < r->L) return 0;
47     if(ql <= r->L && r->R <= qr) return r->sum;
48     return qry(r->lc, ql, qr) + qry(r->rc, ql, qr);
49 }

```

15.4 Time Segment Tree

```

1 // #include Union-find
2 bool unite(int a, int b, vector<pair<int, int>> &update) {
3     a = find(a), b = find(b);
4     if(a == b) return false;
5     if(sz[a] > sz[b]) swap(a, b);
6     update.push_back(make_pair(a, p[a]));
7     sz[b] += sz[a]; p[a] = b;
8     return true;
9 }
10 struct Edge {
11     int a, b;
12     int start, end;
13     Edge() {}
14     Edge(int _a, int _b, int _start) {
15         a = _a, b = _b;
16         start = _start, end = k + 1;
17     }
18 };
19 vector<Edge> edge;
20 map<pair<int, int>, int> indx;
21 void add_start(int a, int b, int start) {
22     if(a > b) swap(a, b);
23     indx[make_pair(a, b)] = (int)edge.size();
24     edge.push_back(Edge(a, b, start));
25 }
26 void add_end(int a, int b, int end) {
27     if(a > b) swap(a, b);
28     int e = indx[make_pair(a, b)];
29     edge[e].end = end;
30 }
31 bool inside(int L1, int R1, int L2, int R2) {
32     // [L1, R1] cover [L2, R2]
33     return (L1 <= L2) && (R2 <= R1);
34 }
35 bool intersect(int L1, int R1, int L2, int R2) {

```

```

36 if(L1 > L2) {
37     swap(L1, L2);
38     swap(R1, R2);
39 }
40 return (R1 > L2);
41 }
42 map<pair<int, int>, vector<Edge>> edge_subset;
43 void add_range(int L, int R, const Edge &e) {
44     if(inside(e.start, e.end, L, R)) {
45         edge_subset[make_pair(L, R)].push_back(e);
46         return;
47     }
48     if(!intersect(L, R, e.start, e.end)) return;
49     int M = (L + R) / 2;
50     add_range(L, M, e);
51     add_range(M, R, e);
52 }
53 void build_range() {
54     for(auto e : edge)
55         add_range(0, k + 1, e);
56 }
57 int ans[N];
58 void solve(int L, int R, int cur_ans) {
59     vector<pair<int, int>> update;
60     int M = (L + R) / 2;
61     for(auto &e : edge_subset[make_pair(L, R)]) {
62         if(inside(e.start, e.end, L, R)) {
63             if(unite(e.a, e.b, update))
64                 cur_ans--;
65         }
66     }
67     if(L == R - 1) {
68         ans[L] = cur_ans;
69     } else {
70         solve(L, M, cur_ans);
71         solve(M, R, cur_ans);
72     }
73     for(int i = (int)update.size() - 1; i >= 0; i--) {
74         auto P = update[i];
75         p[P.first] = P.second;
76     }
77 }
78 int main() {
79     build_dsu();
80     add_start(a, b, i); // add (a, b) at i
81     add_end(a, b, i); // del (a, b) at i
82     build_range();
83     solve(0, k + 1, n);
84 }

```

16 Shortest Path

16.1 Bellman-Ford

```

1 // Edge = {a->b, w}
2 vector<Edge> edge;
3 ll dis[N], n, m;
4 ll Bellman_Ford(int s, int t) {
5     // if return INF, doesn't exist shortest path (s -> t)
6     memset(dis, 0x3f, sizeof(dis));
7     dis[s] = 0;
8     int times = 2 * n;
9     for(int i = 1; i <= times; i++) {
10         for(auto e : edge) {
11             int a = e.a, b = e.b; ll w = e.w;
12             if (dis[a] == INF) continue;
13             if (dis[b] > dis[a] + w) {
14                 dis[b] = dis[a] + w;
15                 if (i >= n) {
16                     if(dis[a] < 0) dis[a] = -INF;
17                     if(dis[b] < 0) dis[b] = -INF;
18                     if (b == t) return INF;
19                 }
20             }
21         }
22     }
23     return dis[t];
24 }

```

16.2 Dijkstra

```

1 priority_queue<pii, vector<pii>, greater<pii>> pq;
2 memset(dis, 0x3f, sizeof(dis));
3 dis[s] = 0; pq.push(make_pair(0, s));
4 while(!pq.empty()) {
5     auto P = pq.top(); pq.pop();
6     int x = P.second; ll _dis = P.first;
7     if(dis[x] < _dis) continue;
8     for(auto &[v, w] : adj[x]) {

```

```

9         if(dis[v] > dis[x] + w) {
10             dis[v] = dis[x] + w;
11             pq.push(make_pair(dis[v], v));
12         }
13     }
14 }

```

16.3 Floyd Warshall

```

1 void Floyd_Warshall() {
2     // init: dis[i][j] = INF, dis[i][i] = 0
3     // add_edge: dis[a][b] = min(dis[a][b], w)
4     for(int k = 1; k <= n; k++) {
5         for(int i = 1; i <= n; i++) {
6             for(int j = 1; j <= n; j++) {
7                 dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
8             }
9         }
10     }
11 }

```

16.4 SPFA

```

1 memset(dis, 0x3f, sizeof(dis));
2 dis[s] = 0;
3 queue<int> que; que.push(s);
4 while(!que.empty()) {
5     int x = que.front();
6     in_que[x] = 0; que.pop();
7     for(auto [v, w] : adj[x]) {
8         if(dis[v] > dis[x] + w) {
9             dis[v] = dis[x] + w;
10             if(!in_que[v]) {
11                 in_que[v] = 1;
12                 que.push(v);
13             }
14         }
15     }
16 }

```

17 Z other Technique

17.1 Baby-Step Giant-Step

```

1 // solve a^x = b % M
2 ll Baby_Giant_Step(ll a, ll b, ll M) {
3     ll k = 1, add = 0, g;
4     while ((g = gcd(a, M)) > 1) {
5         if (b == k) return add;
6         if (b % g) return -1;
7         b /= g, M /= g, add++;
8         k = (k * a / g) % M;
9     }
10    ll RT = sqrt(M)+1, aRT = fp(a, RT, M);
11    unordered_map<ll, ll> mp;
12    for (ll i = 0, cur = b; i <= RT; i++) {
13        mp[cur] = i;
14        cur = (cur * a) % M;
15    }
16    for (ll i = 1, cur = k; i <= M / RT + 1; i++) {
17        cur = (cur * aRT) % M;
18        if (mp.find(cur) != mp.end()) return RT * i - mp[cur] + add;
19    }
20    return -1;
21 }

```

17.2 CDQ

```

1 void CDQ(int l, int r) {
2     if (l == r) return;
3     int m = (l + r) / 2;
4     CDQ(l, m); CDQ(m + 1, r);
5     sort(arr + l, arr + m + 1, cmpB);
6     sort(arr + m + 1, arr + r + 1, cmpB);
7     int i = l, j = m + 1;
8     while (j <= r) {
9         while (i <= m && arr[i].b <= arr[j].b) {
10             BIT.Add(arr[i].c, arr[i].cnt);
11             i++;
12         }
13         arr[j].res += BIT.Ask(arr[j].c);
14         j++;
15     }
16 }

```

```

15     }
16     for (int k = 1; k < i; k++) BIT.Add(arr[k].c, -arr[k].cnt);
17     return;
18 }

```

17.3 Meet in the Middle

```

1 map<ll, int> subset_sum_cnt;
2 void DFS1(int level, ll sum) {
3     if(sum > x) return;
4     if(level == n / 2) {
5         subset_sum_cnt[sum]++;
6         return;
7     }
8     DFS1(level + 1, sum + arr[level]);
9     DFS1(level + 1, sum);
10 }
11 vector<int> sum_list;
12 void DFS2(int level, ll sum) {
13     if(sum > x) return;
14     if(level == n) {
15         sum_list.push_back(sum);
16         return;
17     }
18     DFS2(level + 1, sum + arr[level]);
19     DFS2(level + 1, sum);
20 }
21 ll solve() {
22     DFS1(0, 0); DFS2(n / 2, 0);
23     ll ans = 0;
24     for(auto s : sum_list)
25         ans += subset_sum_cnt[x - s];
26     return ans;
27 }

```

17.4 Quadratic Congruence

```

1 ll w, a; // selected by random
2 struct qr {ll r, q}; // r + q * sqrt(w)
3 qr qr_muti(qr u, qr v, ll p) {
4     qr res;
5     res.r = (u.r * v.r % p + (u.q * v.q) % p * w) % p;
6     res.q = (u.r * v.q % p + u.q * v.r) % p;
7     return res;
8 }
9 qr qr_fp(qr x, ll y, ll p) {
10    qr res;
11    res.r = 1, res.q = 0;
12    while(y) {
13        if(y & 1) res = qr_muti(res, x, p);
14        y >>= 1;
15        x = qr_muti(x, x, p);
16    }
17    return res;
18 }
19 ll p;
20 ll Led(ll n) {
21     // if return p - 1 unsolveable
22     // else if return 1 or 0 solveable
23     return fp(n, (p - 1) / 2, p);
24 }
25 ll QC() {
26     do {
27         a = rand();
28         w = a * a - x;
29         w = (w % p + p) % p;
30     } while (Led(w) != p - 1);
31     qr base;
32     base.r = a, base.q = 1;
33     qr ans = qr_fp(base, (p + 1) / 2, p);
34     ans.r = (ans.r % p + p) % p;
35     return ans.r
36 }

```

ACM ICPC Team

Reference - c0mpile_error

Contents

1 A Reference	1	8.6 Suffix Array + LCP	7
1.1 misc	1	8.7 Suffix Automaton	8
1.2 python	1	8.8 Trie	8
2 B Graph Theory	1	8.9 Z algorithm	8
2.1 BCC Tarjan	1	9 Flow	8
2.2 Bipartite Matching	1	9.1 Max-Flow (Dinic)	8
2.3 Euler Path	1	9.2 Min Cost Max-Flow	9
2.4 Kuhn Munkres	2	10 G Geometric Computing	9
2.5 König's theorem	2	10.1 Convex Hull	9
2.6 Max Clique	2	10.2 Half-Plane Intersection	9
2.7 Topological sort	2	10.3 Heart	9
3 C Dynamic Programing	2	10.4 Line Segment Intersection	10
3.1 Classic DP Problem Set	2	10.5 Minimum Enclosing Circle	10
3.2 Digit DP	3	10.6 Minimum Euclidean Distance	10
3.3 Subset DP	3	10.7 Pick's Therom	10
3.4 Sum Over Subsets	3	10.8 Point and Cross	10
4 Combination	3	10.9 Points in Convex Hull	10
4.1 Burnside's Lemma	3	10.10 PointSegDist	11
4.2 Catalan Number	3	10.11 Polygon Area	11
4.3 Fast Matrix Exponentiation	3	11 H Number Theorem	11
5 D Data Structure	3	11.1 Chinese Remainder Theorem	11
5.1 1D-BIT	3	11.2 Coordinate Compression	11
5.2 2D-BIT	4	11.3 Determinant and Gaussian Elimination	11
5.3 Mo's algorithm	4	11.4 Extended Euclidean	11
5.4 Sparse Table	4	11.5 Fast Power and BigInt Multiplication	11
5.5 Treap	4	11.6 FFT	11
5.6 Union-Find	4	11.7 Josephus	12
6 DP optimization	4	11.8 Nim Game	12
6.1 Divide and Conquer	4	12 MST	12
6.2 DP Optimization Note	5	12.1 Kruskal	12
6.3 Knuth Optimization	5	12.2 Prim	12
6.4 LiChao	5	13 Prime and Composite	12
7 E Tree algorithm	5	13.1 Factorize and Phi	12
7.1 Cetroid Decomposition	5	13.2 Miller Rabin	12
7.2 Change Root DP	5	13.3 Mobius	12
7.3 DSU on Tree	6	13.4 Pollord rho	13
7.4 Heavy-Light Decomposition	6	13.5 Prime Table	13
7.5 LCA	6	14 SCC	13
7.6 Tree Diameter	6	14.1 2-SAT	13
7.7 Tree Traversals Array	6	14.2 Kosaraju	13
8 F String	6	14.3 Tarjan	13
8.1 Aho-Corasick	6	15 Segment Tree	13
8.2 KMP	7	15.1 Area of Union	13
8.3 Manacher	7	15.2 Lazy Segment Tree	14
8.4 Minimal Rotation	7	15.3 Persistent Segment Tree	14
8.5 String Hash	7	15.4 Time Segment Tree	14
		16 Shortest Path	15
		16.1 Bellman-Ford	15
		16.2 Dijkstra	15
		16.3 Floyd Warshall	15
		16.4 SPFA	15
		17 Z other Technique	15
		17.1 Baby-Step Giant-Step	15
		17.2 CDQ	15
		17.3 Meet in the Middle	16
		17.4 Quadratic Congruence	16

ACM ICPC Judge Test - c0mpile_error

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6     const size_t KB = 1024;
7     const size_t MB = KB * 1024;
8     const size_t GB = MB * 1024;
9
10    size_t block_size, bound;
11    void stack_size_dfs(size_t depth = 1) {
12        if (depth >= bound)
13            return;
14        int8_t ptr[block_size]; // 若無法編譯將 block_size 改成常數
15        memset(ptr, 'a', block_size);
16        cout << depth << endl;
17        stack_size_dfs(depth + 1);
18    }
19
20    void stack_size_and_runtime_error(size_t block_size, size_t
        bound = 1024) {
21        system_test::block_size = block_size;
22        system_test::bound = bound;
23        stack_size_dfs();
24    }
25
26    double speed(int iter_num) {
27        const int block_size = 1024;
28        volatile int A[block_size];
29        auto begin = chrono::high_resolution_clock::now();
30        while (iter_num--)
31            for (int j = 0; j < block_size; ++j)
32                A[j] += j;
33        auto end = chrono::high_resolution_clock::now();
34        chrono::duration<double> diff = end - begin;
35        return diff.count();
36    }
37

```

```

38 void runtime_error_1() {
39     // Segmentation fault
40     int *ptr = nullptr;
41     *(ptr + 7122) = 7122;
42 }
43
44 void runtime_error_2() {
45     // Segmentation fault
46     int *ptr = (int *)memset;
47     *ptr = 7122;
48 }
49
50 void runtime_error_3() {
51     // munmap_chunk(): invalid pointer
52     int *ptr = (int *)memset;
53     delete ptr;
54 }
55
56 void runtime_error_4() {
57     // free(): invalid pointer
58     int *ptr = new int[7122];
59     ptr += 1;
60     delete[] ptr;
61 }
62
63 void runtime_error_5() {
64     // maybe illegal instruction
65     int a = 7122, b = 0;
66     cout << (a / b) << endl;
67 }
68
69 void runtime_error_6() {
70     // floating point exception
71     volatile int a = 7122, b = 0;
72     cout << (a / b) << endl;
73 }
74
75 void runtime_error_7() {
76     // call to abort.
77     assert(false);
78 }
79
80 } // namespace system_test
81
82 #include <sys/resource.h>
83 void print_stack_limit() { // only work in Linux
84     struct rlimit l;
85     getrlimit(RLIMIT_STACK, &l);
86     cout << "stack_size = " << l.rlim_cur << " byte" << endl;
87 }

```